

SMART CAMPUS EVENT MANAGEMENT AND REGISTRATION SYSTEM USING SPRING BOOT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

G RAAJIDHA (VTU26488)

*10211CS224
Full Stack Application Development
Slot : E1-B2*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Smart Campus Event Management and Registration System using Spring Boot" by G RAAJIDHA (VTU26488) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(G RAAJIDHA)

Date: 06/05/2026

ABSTRACT

The Campus Event Management System is a full-stack web application designed to streamline event discovery, management, and registration within a campus. Built using Spring Boot for the backend and Thymeleaf with Bootstrap 5 for the frontend, it provides a responsive and user-friendly interface for students and a centralized dashboard for administrators. Key features include secure admin authentication using Spring Security, event browsing with pagination, and automated email confirmations. The system uses a MySQL database with Spring Data JPA to ensure reliable data storage and maintain relationships between events and registrations. It also demonstrates the use of DTO mapping, global exception handling, and modern web design principles to deliver a smooth event registration experience.

Keywords: Full-Stack Development, Spring Boot, Thymeleaf, Java, MySQL, Event Management, Spring Security.

LIST OF FIGURES

1.1	System Overview Diagram	3
3.1	Backend Architecture	9
5.1	Admin Login Page	18
5.2	Event Discovery Portal	19
5.3	Admin Dashboard	20
5.4	Event Registration Interface	21

LIST OF TABLES

5.1	Application Feature Comparison	17
7.1	Mapping of Project to Complex Engineering Activities (CEA)	26
7.2	SDG Mapping for the Project	27

LIST OF ACRONYMS AND ABBREVIATIONS

DTO	Data Transfer Object
CRUD	Create, Read, Update, Delete
MVC	Model View Controller
JPA	Java Persistence API
UI	User Interface
ORM	Object Relational Mapping
SQL	Structured Query Language
DI	Dependency Injection

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6

3.2	Backend Implementation	7
3.3	Database Implementation	8
4	METHODOLOGIES	10
4.1	Project Architecture	10
4.2	DataFlow Diagram	10
4.3	Module 1: Administrator Authentication	11
4.4	Module 2: Event Management & Discovery	12
4.5	Module 3: Registration & Notification System	13
5	RESULTS AND DISCUSSIONS	16
5.0.1	Admin Login Page	18
5.0.2	Event Discovery Portal	19
5.0.3	Admin Dashboard	20
5.0.4	Event Registration Page	21
6	CONCLUSION AND FUTURE ENHANCEMENTS	22
6.1	Conclusion	22
6.2	Future Enhancements	23
6.3	Login Page	23
6.4	Login Controller	24
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	25

7.1	Mapping of the Project to Complex Engineering Problem (CEP)	25
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	26
7.3	SDG Mapping (CEP Section)	27

References	28
-------------------	-----------

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid digitalization of services has transformed how people interact with events, shifting from manual ticketing systems to centralized online platforms. The Campus Event Management System is developed to address the need for a reliable, scalable, and intuitive system that bridges the gap between event organizers and students[1]. By leveraging modern Java-based web technologies, this platform facilitates seamless event discovery, secure student registration, and hassle-free management processes[2].

1.2 Aim of the Project

The primary aim of this project is to design and develop a robust web-based application that allows students to easily browse and register for various departmental and institutional events. Additionally, it

aims to provide administrators with a powerful, secure dashboard to create, monitor, update, and delete event listings while tracking registration records efficiently.

1.3 Scope of the Project

The scope of the project encompasses:

- A responsive user-facing frontend for browsing events with pagination and search functionalities, and registering for events.
- A secure administrative dashboard leveraging Spring Security for comprehensive CRUD operations on events and tracking registrations.
- An automated email notification system utilizing Spring Boot Starter Mail to send registration confirmations to students.
- A resilient backend utilizing Spring Boot's MVC architecture to handle business logic, DTO mapping, and global exception handling.
- A relational database schema utilizing Spring Data JPA (Hibernate) to efficiently store administrator credentials, event details, and registration transactions.

1.4 Framework

The application is built using a modern Java ecosystem tailored for scalable web applications:

- **Frontend:** Thymeleaf templates enhanced with Bootstrap 5 for responsiveness.
- **Backend:** Java 17, Spring Boot 3.x, Spring Web MVC, Spring Security, Spring Boot Mail.
- **Database:** MySQL relational database accessed via Spring Data JPA.

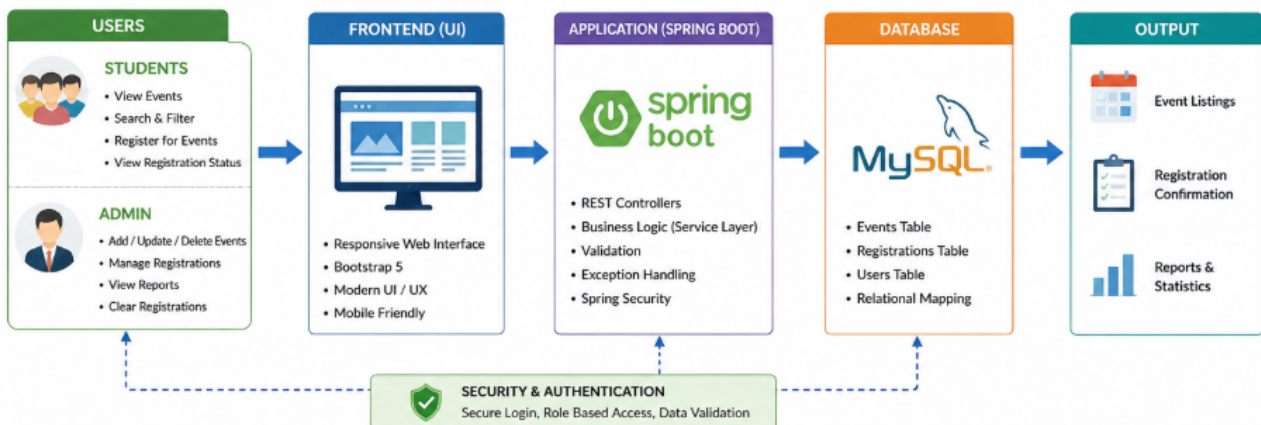


Figure 1.1: System Overview Diagram

The system overview shows how users interact with the application. The frontend handles user input, the backend processes logic using Spring Boot, and the database stores event and registration data. All components work together to provide a smooth event management system[3].

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The market for event management platforms features industry leaders such as Eventbrite, Meetup, and BookMyShow.

- **Eventbrite:** Offers extensive tools for organizers but can be highly complex and introduces significant transaction fees, making it less ideal for localized or institutional campus events.
- **Meetup:** Focuses primarily on community-building and recurring group meetups. It lacks the straightforward registration and tracking structure needed for standalone academic or departmental events.
- **BookMyShow:** A dominant player in the entertainment sector, but its architecture is heavily tailored for large-scale cinema and concert events, making it overkill for campus event management.

The proposed Campus Event Management System differentiates itself by offering a lightweight, highly customizable, and cost-effective solution specifically tailored for institutional and departmental events. It eliminates the overhead of enterprise-level platforms while retaining core functionalities[4]. Additionally, the system provides a simplified, mobile-responsive user interface using Bootstrap 5 and a streamlined workflow ensuring ease of use. Unlike existing platforms that may overwhelm users with unnecessary features, this application focuses strictly on essential functionalities such as event creation, registration, pagination, and search capabilities, making it more accessible for academic organizational use.

Furthermore, the application is designed with maintainability in mind, utilizing Java Spring Boot's layered architecture (Controllers, Services, Repositories). This makes the system adaptable to future enhancements, such as payment gateway integrations, while remaining highly performance[5].

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend relies on server-side rendering using Thymeleaf, seamlessly integrated with Spring Boot. Key design dependencies include Bootstrap 5, loaded via CDNs, ensuring modern, responsive UI components without the need for complex Node.js build pipelines[6].

3.1.2 Structure of the Project

The frontend view templates (located in the `src/main/resources/templates` directory) are organized intuitively:

- **fragments/**: Contains reusable UI components such as header, navbar, footer, and alert modals.
- **admin/**: Contains protected views for event management, admin user management, and registration tracking.

- **public/**: Contains user-facing views like event catalogs, search results, and registration forms.

3.1.3 Execution Procedure

The frontend does not require a separate server execution. It runs embedded within the Spring Boot Tomcat server and is compiled and served automatically when the backend server is launched.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built using the robust Spring Boot framework (Java 17). It relies on Maven for dependency management (`pom.xml`). Critical dependencies include `spring-boot-starter-web` for MVC, `spring-boot-starter-data-jpa` for ORM, `spring-boot-starter-security` for authentication, and `spring-boot-starter-mail` for automated emails[7].

3.2.2 Structure of the Project

The backend architecture rigorously follows a multi-tier structure:

- **model/**: Contains JPA entities mapping to database tables (`Event`, `Admin`, `Registration`).
- **dto/**: Data Transfer Objects ensuring separation between database entities and UI payloads.

- **repository/**: Spring Data JPA interfaces for database CRUD operations.
- **service/**: Contains business logic, email processing, and transactional operations.
- **controller/**: Handles incoming HTTP requests and routes them to appropriate views.
- **config/**: Contains security configurations and component setups.
- **exception/**: Global exception handlers using `@ControllerAdvice`.

3.2.3 Execution Procedure

To execute the application:

1. Ensure a MySQL database server is running locally and the database `campus_events` is created.
2. Update the 'application.properties' with the correct database credentials.
3. Open the terminal in the project root directory.
4. Execute `./mvnw spring-boot:run` to start the embedded Tomcat server. The application will be accessible at `http://localhost:8081`.

3.3 Database Implementation

3.3.1 Database Configuration

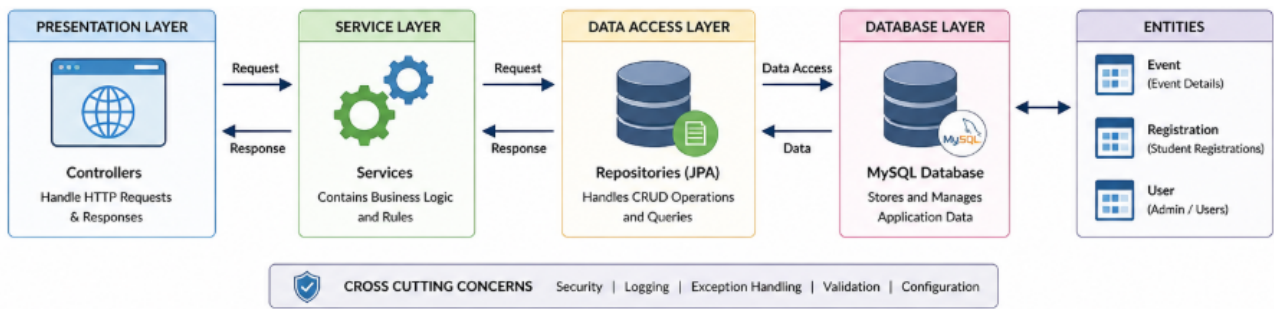


Figure 3.1: Backend Architecture

The application utilizes MySQL for persistent data storage. The connection is managed via HikariCP and Spring Data JPA via JDBC. The connection parameters are specified in the `application.properties` file.

3.3.2 Schema Structure

The schema is relationally designed and automatically generated and updated via Hibernate (`spring.jpa.hibernate.ddl-auto=update`).

3.3.3 Tables Created

- **admin:** Stores administrator credentials securely. Columns include ID, Username, and Password.
- **event:** Stores event details. Columns include ID, Name, Description, Date, Time, Venue, and Total Capacity.
- **registration:** Stores student registration records. Columns include ID, Student Name, Student Email, Registration Date, and a foreign key linking to the event table.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a Monolithic Model-View-Controller (MVC) architecture. Incoming client requests are handled by the Spring DispatcherServlet, which routes them to the appropriate Controller. Controllers interact with Service layers via Dependency Injection, which in turn use Repositories for data access. Processed data is then bound to Thymeleaf views and returned as formatted HTML to the client browser[8].

4.2 DataFlow Diagram

1. The User interacts with the UI to submit a registration form.
2. The browser sends an HTTP POST request to the corresponding Controller mapping.
3. The Controller validates the input data against the DTO. If valid, it

invokes the Service layer.

4. The Service layer executes business logic and interacts with the JPA Repository to save the `Registration` entity.
5. An automated confirmation email is queued and dispatched asynchronously via Spring Boot Mail.
6. The user is redirected to a success view with a confirmation message.

4.3 Module 1: Administrator Authentication

This module is a critical security component that leverages the robust capabilities of Spring Security to rigorously restrict access to sensitive administrative endpoints and management URLs. It establishes a secure gateway utilizing form-based authentication, which intercepts unauthorized requests and seamlessly redirects them to a dedicated login portal. Behind the scenes, the module actively manages user sessions, tracking authenticated states to prevent session hijacking and cross-site request forgery (CSRF) vulnerabilities. User credentials are securely managed and authenticated against backend records, adhering to industry best practices for data protection. Furthermore, the authentication mechanism ensures that only authorized personnel with verified administrative privileges can access the core functionalities of

the system. This includes safeguarding the event creation dashboard, modifying existing event records, and accessing comprehensive registration tracking tools. By implementing stringent access control, the system maintains strict data integrity and operational security, guaranteeing that the management infrastructure remains impenetrable to unauthorized users and automated malicious bots.

4.4 Module 2: Event Management & Discovery

Serving as the operational core of the application, this module provides administrators with a comprehensive suite of tools to execute full CRUD (Create, Read, Update, Delete) operations on event records. Through an intuitive, Bootstrap-powered dashboard, organizers can seamlessly generate new event listings by specifying critical details such as event titles, detailed descriptions, precise scheduling, venue locations, and maximum attendee capacities. Conversely, on the public-facing side, the module delivers an optimized event discovery experience for students. To handle extensive datasets efficiently, it incorporates advanced server-side pagination utilizing Spring Data JPA, ensuring that the user interface remains responsive and lightweight, even when rendering hundreds of active events. Additionally, a robust search mechanism empowers users to filter events dynamically

based on relevant keywords or categories. This dual-faceted approach guarantees that administrators possess the necessary granular control to manage institutional schedules, while simultaneously providing students with a fast, intuitive, and accessible catalog to explore upcoming campus activities without experiencing performance bottlenecks.

4.5 Module 3: Registration & Notification System

This module facilitates the entire student enrollment workflow, serving as the bridge between user intent and successful event participation. When a student initiates a registration request, the system actively performs real-time capacity validation against the backend MySQL database, ensuring that the selected event has not exceeded its predefined maximum attendance limit. Upon successful validation, the module securely records the participant's details, persisting the transactional data via Hibernate ORM to maintain relational integrity between the student and the event. Immediately following a successful database commit, the system leverages Spring Boot Starter Mail to asynchronously dispatch a personalized booking confirmation email to the user via an integrated SMTP service, providing immediate digital verification of their reserved slot. To enhance system resilience, the module incorporates comprehensive global exception handling uti-

lizing the `@ControllerAdvice` annotation. This sophisticated error-management architecture guarantees that any registration anomalies—such as attempting to book a fully-booked event or submitting invalid email formats—are gracefully intercepted. Consequently, users are presented with clear, actionable, and user-friendly alert notifications instead of technical stack traces.

Advantages of this Project

- **Robustness:** The statically typed nature of Java and the comprehensive ecosystem of Spring Boot provide a highly reliable foundation.
- **SEO Friendly:** Server-side rendering with Thymeleaf provides fully populated HTML directly to the browser, significantly enhancing search engine optimization.
- **Maintainability:** The rigid enforcement of the MVC pattern and DTOs prevents spaghetti code and facilitates future upgrades.

Disadvantages

- **Monolithic Coupling:** Unlike decoupled frontend/backend applications, the UI and API logic are tightly coupled in the same deployment unit, making it slightly harder to scale independently.

- **Server Load:** Server-side rendering consumes more server CPU and memory compared to delivering a static Single Page Application (SPA).

This project emphasizes clean coding practices such as Dependency Injection and aspect-oriented exception handling. The Bootstrap 5 integration ensures the application remains highly accessible across diverse devices[9].

Chapter 5

RESULTS AND DISCUSSIONS

The Campus Event Management System successfully fulfills its functional requirements. Students can intuitively navigate the platform, utilize the search and pagination tools to discover events, and register seamlessly. Administrators possess reliable oversight over event listings via a secured dashboard[10].

The integration of Thymeleaf provides fast, responsive page loads. The Spring Boot backend efficiently processes form submissions, and the MySQL database accurately maintains data integrity. Email confirmations dispatch reliably.

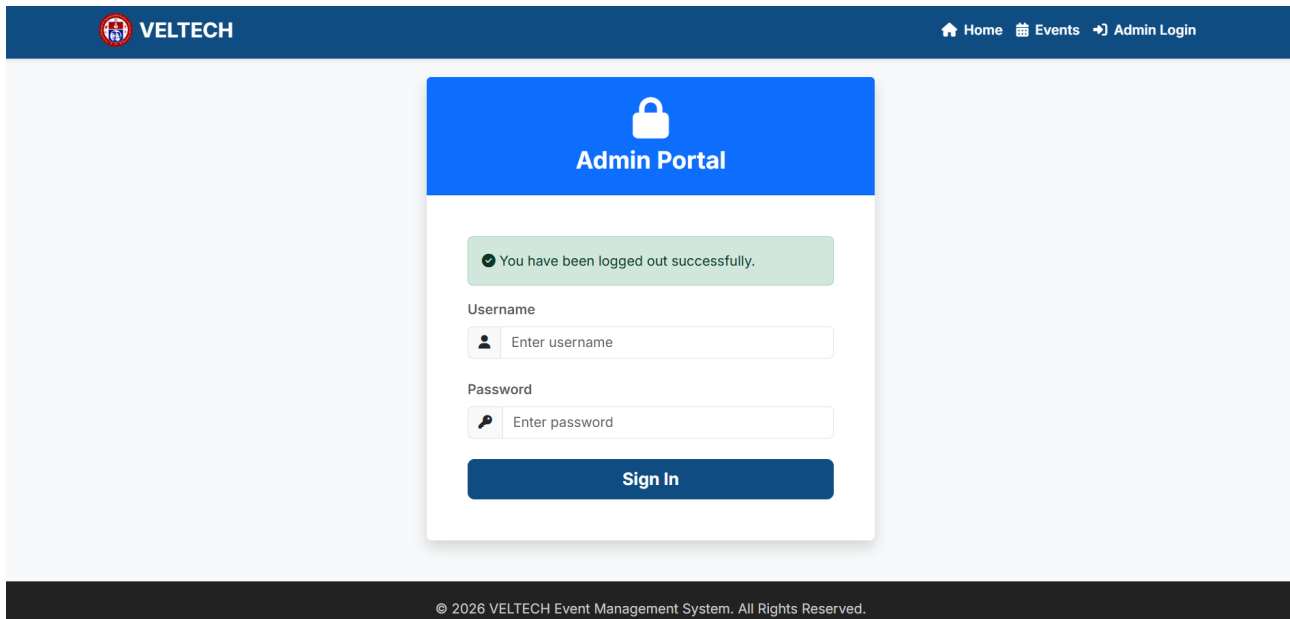
Feature	Eventbrite	BookMyShow	Proposed App
Custom Branding	Limited	Limited	High
Transaction Fees	High	Moderate	None (Self-Hosted)
Setup Complexity	High	Moderate	Low
Target Audience	Commercial Events	Entertainment Sector	Institutional / Departmental
Data Ownership	Third-Party	Third-Party	Fully Owned
Architecture	Microservices	Microservices	Monolithic MVC
Cost Efficiency	Low	Moderate	High

Table 5.1: Application Feature Comparison

As illustrated in the Application Feature Comparison table above, the proposed Campus Event Management System provides significant advantages for institutional use over existing commercial platforms. While applications like Eventbrite and BookMyShow excel in the commercial and entertainment sectors, they often impose high transaction fees, limited branding capabilities, and complex setups.

The developed application is currently deployed in a local environment (localhost:8081) and includes the following views:

5.0.1 Admin Login Page



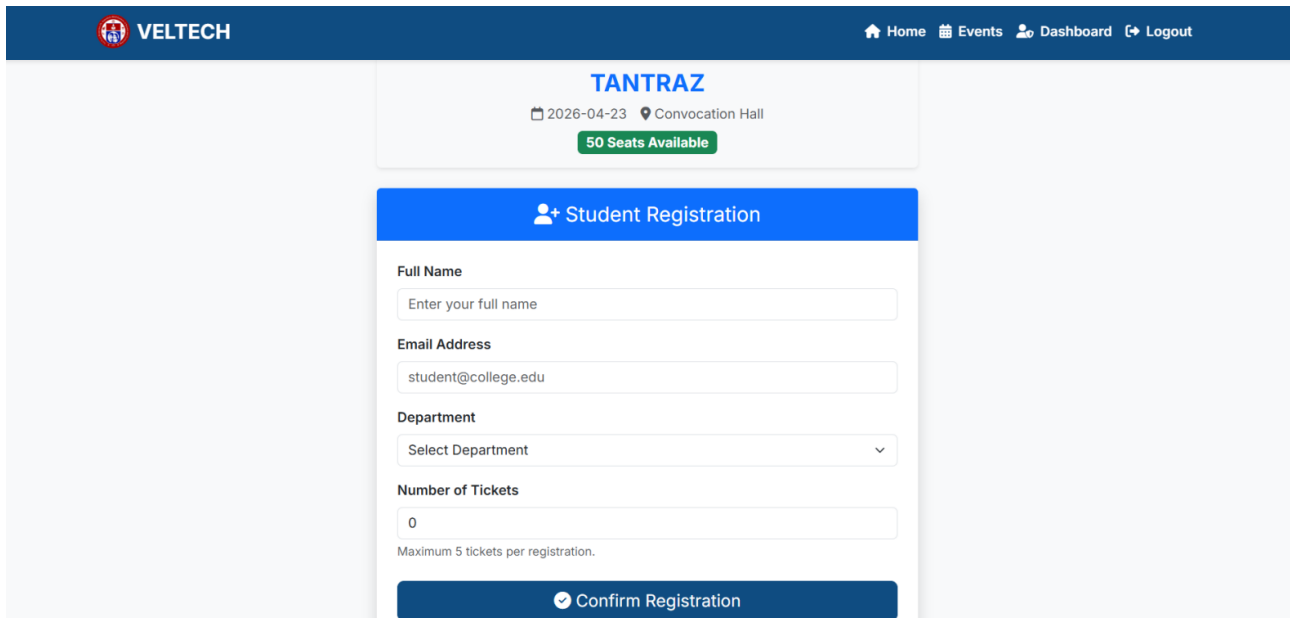
The screenshot displays the Admin Login Page of the VELTECH Event Management System. The page features a dark blue header with the VELTECH logo on the left and navigation links for Home, Events, and Admin Login on the right. The main content area is light gray, centered around a white login form with a blue header. The form's blue header contains a white padlock icon and the text 'Admin Portal'. Below this, a green message box states 'You have been logged out successfully.' The form includes two input fields: 'Username' with a placeholder 'Enter username' and 'Password' with a placeholder 'Enter password'. A dark blue 'Sign In' button is positioned at the bottom of the form. The footer is a dark gray bar with the copyright notice '© 2026 VELTECH Event Management System. All Rights Reserved.'

Figure 5.1: Admin Login Page

The figure above illustrates the secure login portal designed exclusively for administrators. Built with Bootstrap 5, the interface provides a clean, responsive layout where authorized personnel must authenticate themselves using valid credentials.

5.0.2 Event Discovery Portal

Displays available events to students with server-side pagination and search capabilities.



The screenshot displays the Event Discovery Portal interface. At the top, a dark blue header bar contains the VELTECH logo on the left and navigation links (Home, Events, Dashboard, Logout) on the right. Below the header, a light gray box highlights the event details: **TANTRAZ**, dated 2026-04-23, located at Convocation Hall, with a green badge indicating **50 Seats Available**. The main content area features a blue header for the **Student Registration** form. The form includes fields for Full Name (placeholder: Enter your full name), Email Address (placeholder: student@college.edu), Department (a dropdown menu with 'Select Department'), and Number of Tickets (placeholder: 0). A note below the tickets field states 'Maximum 5 tickets per registration.' At the bottom of the form is a blue button with a checkmark icon and the text 'Confirm Registration'.

Figure 5.2: Event Discovery Portal

The figure above displays the primary user-facing interface where students can browse upcoming campus events. The portal features a dynamic search bar for filtering events by name or category, and implements server-side pagination to handle large volumes of data without browser lag.

5.0.3 Admin Dashboard

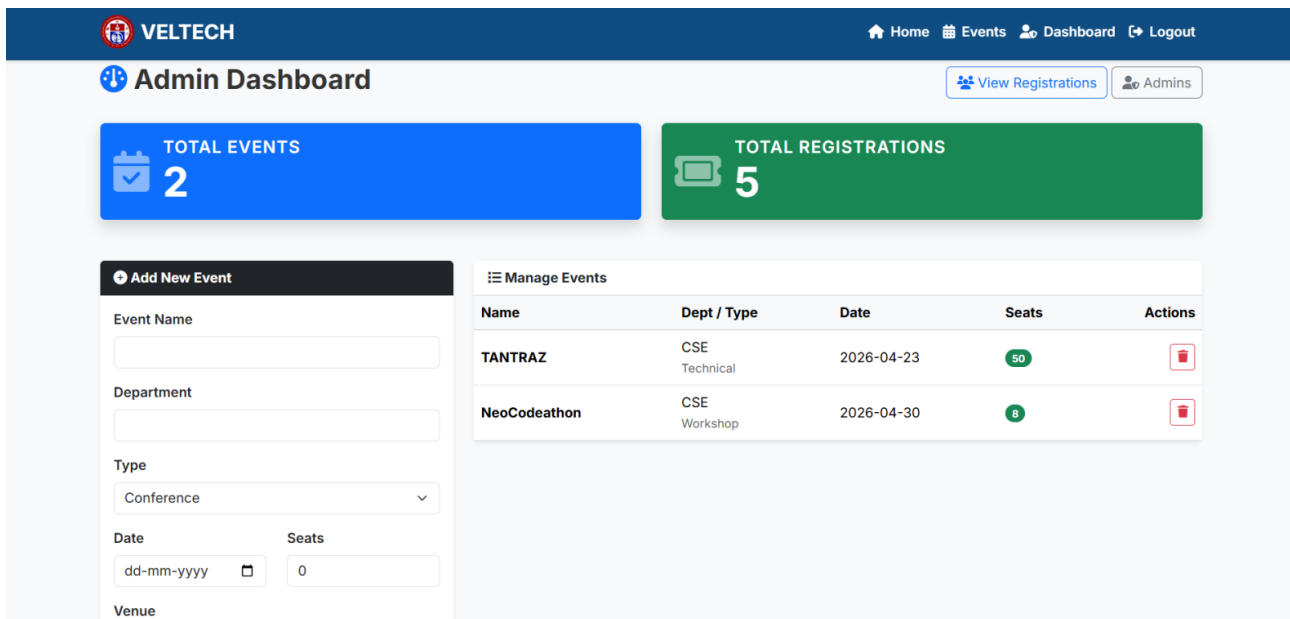


Figure 5.3: Admin Dashboard

The figure above showcases the comprehensive Admin Dashboard, serving as the central control panel for event management. From this interface, administrators can easily perform full CRUD operations—creating new events, modifying existing details, or deleting outdated listings. The dashboard also provides real-time tracking of registration metrics, allowing organizers to monitor attendance limits and manage student enrollment efficiently from a unified, easily accessible view.

5.0.4 Event Registration Page

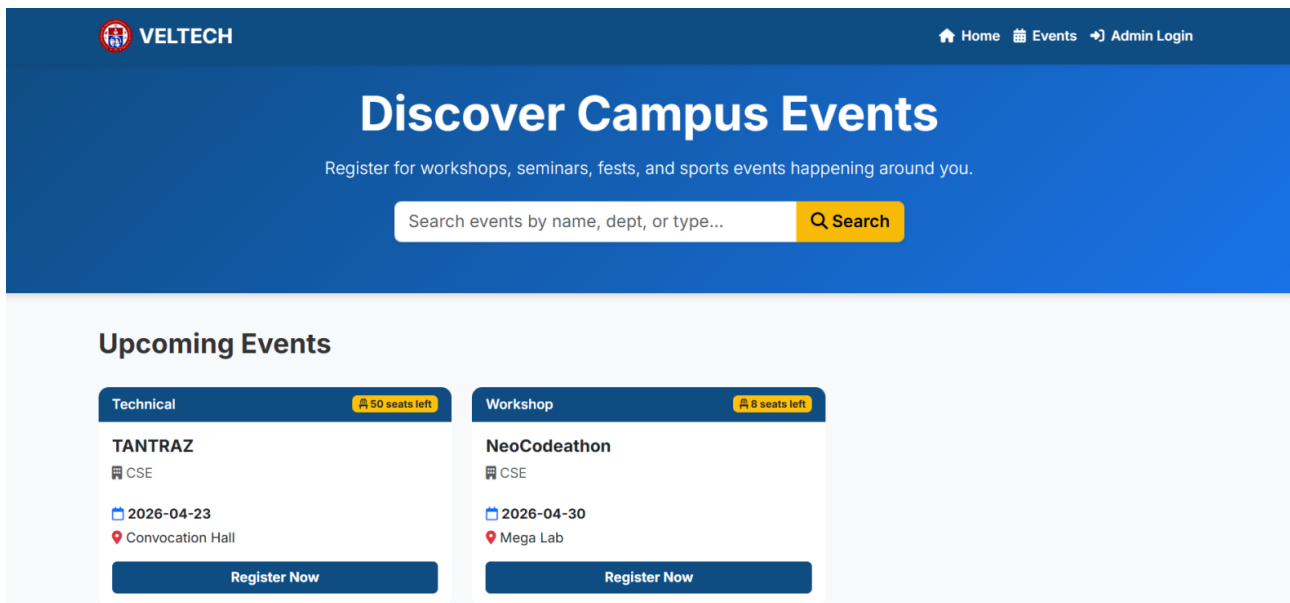


Figure 5.4: Event Registration Interface

The figure above depicts the dedicated event registration form utilized by students to secure their participation. The form incorporates rigorous DTO-based validation to ensure all submitted data, such as the student's email and identification number, is accurately formatted. Upon submission, the system actively performs a real-time capacity check against the backend database before confirming the booking and triggering an automated SMTP confirmation email to the participant.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Campus Event Management System demonstrates the continued effectiveness of server-side rendered architectures using modern Java frameworks. By integrating Spring Boot, Thymeleaf, Spring Security, and MySQL, the system provides a robust, fast, and accessible platform for managing institutional events.

The application successfully implements complex backend functionalities like DTO mappings, transactional safety, and global error handling, while offering an attractive frontend powered by Bootstrap5.

6.2 Future Enhancements

- **RESTful API Transition:** Decouple the application by exposing REST endpoints and migrating the frontend to a SPA framework like React or Angular.
- **QR Code Check-ins:** Generate dynamic QR codes and email them to students for fast physical check-in at event venues.
- **Advanced Analytics:** Implement charting libraries in the admin dashboard to visualize event popularity and attendance trends.
- **Dockerization:** Containerize the application using Docker to streamline deployment processes to cloud environments.

6.3 Login Page

```
1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4     <title>Admin Login</title>
5     <link rel="stylesheet" href="/css/style.css">
6 </head><body>
7 <h2>Admin Login</h2>
8 <div th:if="${param.error}">
9     <p style="color:red;">Invalid username or password</p>
10 </div>
11 <form th:action="@{/login}" method="post">
12     <label>Username:</label>
13     <input type="text" name="username" required>
14     <br><br>
15     <label>Password:</label>
```

```

16     <input type="password" name="password" required>
17     <br><br>
18     <button type="submit">Login </button>
19 </form></body>
20 </html>

```

6.4 Login Controller

```

1 package com.campus.eventmanagement.config;
2 import org.springframework.context.annotation.Bean;
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
5 import org.springframework.security.web.SecurityFilterChain;
6 import org.springframework.security.core.userdetails.*;
7 import org.springframework.security.crypto.password.NoOpPasswordEncoder;
8 @Configuration
9 public class SecurityConfig {
10     @Bean
11     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
12         http .authorizeHttpRequests(auth -> auth.requestMatchers("/", "/register",
13             "/**"/css/**").permitAll().requestMatchers("/admin/**").hasRole("ADMIN")
14             .anyRequest().authenticated())
15             .formLogin(form -> form.loginPage("/login").defaultSuccessUrl("/admin", true).permitAll()).logout(logout -> logout.permitAll());
16         return http.build();
17     }@Bean
18     public UserDetailsService userDetailsService() {
19         UserDetails admin = User.withUsername("admin").password("admin123").roles("ADMIN").build();
20         return new InMemoryUserDetailsManager(admin);
21     }@Bean
22     public static NoOpPasswordEncoder passwordEncoder() {
23         return (NoOpPasswordEncoder) NoOpPasswordEncoder.getInstance();
24     }
25 }

```

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, REST APIs, and frontend technologies.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security by ensuring easy access while maintaining secure authentication.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves system design, schema creation, validation handling,
WP4	Familiarity	Partially new problems	WK4, WK8	Includes real-time updates, user interaction, and analytics.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for tracking and filtering.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	Addresses needs of students.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates UI, backend, database, validation, and security modules.

Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases.
EA2	Level of Interactions	Complex interactions between modules	Handles event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences	Impact on users and society	Improves efficiency and reduces manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and analytics into one system.

Table 7.1: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Facilitates student participation in academic events and enhances learning.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation through centralized systems.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives and resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent system operations.
SDG 12	Responsible Consumption and Production	Reduces paper usage through online management.

Table 7.2: SDG Mapping for the Project

REFERENCES

- [1] Eventbrite Platform. A widely used event management platform that provides tools for event creation, ticketing, and analytics. Available at: <https://www.eventbrite.com>
- [2] Meetup Platform. A community-based event platform that allows users to join groups and participate in events. Available at: <https://www.meetup.com>
- [3] BookMyShow Platform. An online ticket booking system primarily focused on entertainment events like movies and shows. Available at: <https://in.bookmyshow.com>
- [4] Spring Boot Documentation. Official documentation providing guidance for building production-ready Java applications using the Spring Boot framework. Available at: <https://spring.io/projects/spring-boot>
- [5] Spring Framework Documentation. Comprehensive documentation for developing enterprise-level Java applications using Spring technologies. Available at: <https://spring.io/projects/>

spring-framework

- [6] Spring Security Documentation. Provides detailed information about authentication and authorization mechanisms in web applications. Available at: <https://docs.spring.io/spring-security/reference/index.html>
- [7] Thymeleaf Documentation. A modern server-side Java template engine used for building dynamic and responsive web applications. Available at: <https://www.thymeleaf.org>
- [8] MySQL Documentation. Official reference for MySQL database covering installation, configuration, and database management. Available at: <https://dev.mysql.com/doc/>
- [9] Oracle Java Documentation. Provides detailed information about Java programming language, APIs, and development tools. Available at: <https://docs.oracle.com/en/java/>
- [10] Bootstrap Framework Documentation. A popular frontend framework used for designing responsive and modern web interfaces. Available at: <https://getbootstrap.com>